

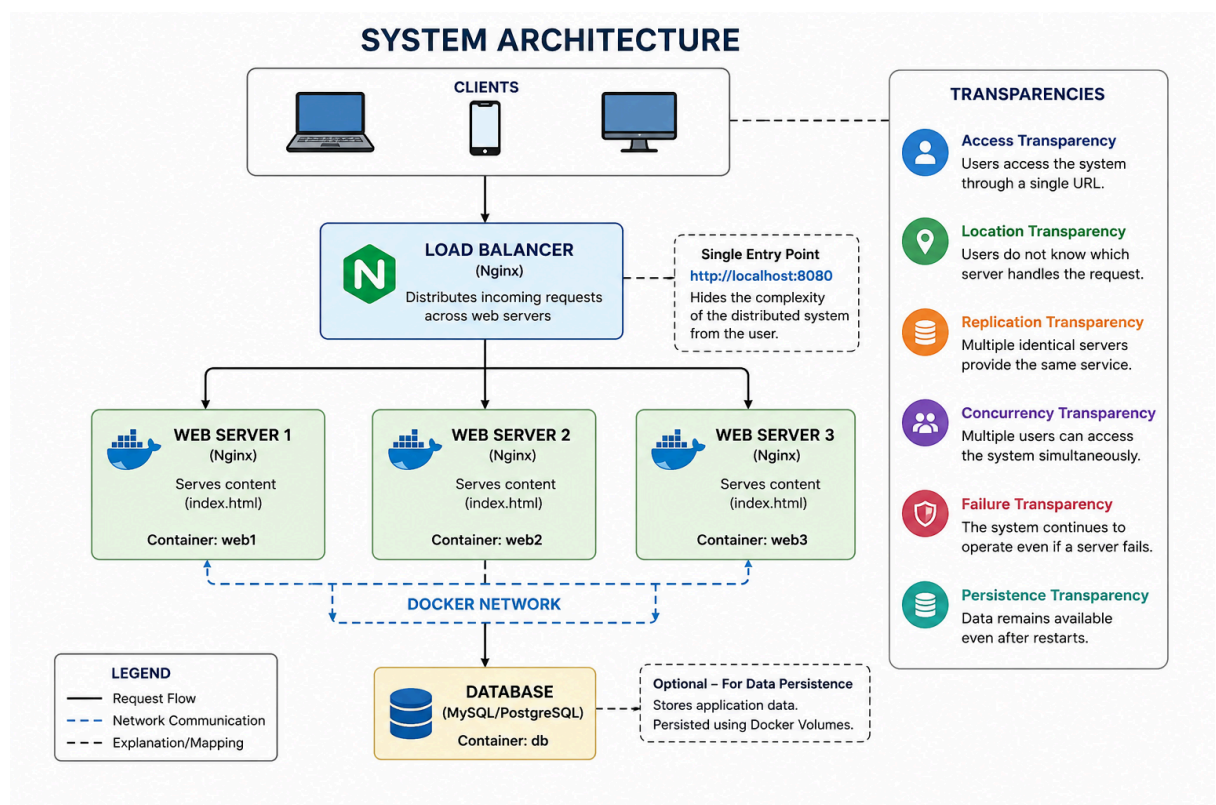
Lab

Implementing Distributed System Transparencies Using Docker Containers

Learning Outcomes

Upon successful completion of this lab, students should be able to:

- Deploy distributed services using Docker
- Configure container networking
- Implement service replication
- Configure load balancing
- Demonstrate distributed system transparencies
- Analyze fault tolerance mechanisms
- Explain persistence in distributed systems



Part 1 – Install Docker

Verify Docker installation.

```
docker --version
```

Verify Docker Compose.

```
docker compose version
```

Part 2 – Create Project Structure

Create the following directories:

```
mkdir distributed-lab  
cd distributed-lab
```

```
mkdir web1  
mkdir web2  
mkdir web3  
mkdir nginx
```

Part 3 – Create Web Pages

Web Server 1

Create:

```
nano web1/index.html  
<!DOCTYPE html>  
<html>  
<head>  
<title>Server 1</title>  
</head>  
<body>  
<h1>Response from Web Server 1</h1>  
</body>  
</html>
```

Web Server 2

Create:

```
nano web2/index.html
<!DOCTYPE html>
<html>
<head>
  <title>Server 2</title>
</head>
<body>
  <h1>Response from Web Server 2</h1>
</body>
</html>
```

Web Server 3

Create:

```
nano web3/index.html
<!DOCTYPE html>
<html>
<head>
  <title>Server 3</title>
</head>
<body>
  <h1>Response from Web Server 3</h1>
</body>
</html>
```

Part 4 – Create Nginx Load Balancer Configuration

Create:

```
nano nginx/nginx.conf
events {}

http {

  upstream backend {
```

```
server web1:80;
server web2:80;
server web3:80;

}

server {

    listen 80;

    location / {

        proxy_pass http://backend;

    }
}
}
```

Part 5 – Create Docker Compose File

Create:

```
nano docker-compose.yml
services:

web1:
  image: nginx
  container_name: web1
  volumes:
    - ./web1/index.html:/usr/share/nginx/html/index.html

web2:
  image: nginx
  container_name: web2
  volumes:
    - ./web2/index.html:/usr/share/nginx/html/index.html

web3:
  image: nginx
  container_name: web3
```

volumes:

- ./web3/index.html:/usr/share/nginx/html/index.html

loadbalancer:

image: nginx

container_name: loadbalancer

ports:

- "8080:80"

volumes:

- ./nginx/nginx.conf:/etc/nginx/nginx.conf

depends_on:

- web1
- web2
- web3

Part 6 – Deploy the System

Start all containers.

```
docker compose up -d
```

Verify:

```
docker ps
```

Expected:

web1

web2

web3

Loadbalancer

Part 7 – Verify Load Balancing

Open:

<http://localhost:8080>

Refresh multiple times.
Observe responses from:

Web Server 1
Web Server 2
Web Server 3

Part 8 – Container Networking

View network.

docker network ls

Inspect network.

docker network inspect distributed-lab_default

Observe:

- Container IP addresses
- Service names
- Network connectivity

Part 9 – Investigating Distributed System Transparencies

Students must perform the required activities and answer the following questions. For each transparency, explain:

- What changes or configurations are required to achieve the transparency.
- How your implementation demonstrates the transparency.
- Evidence (screenshots, commands, observations) supporting your answer.

Question 1 – Access Transparency

A user should be able to access the distributed application through a single endpoint without needing to know the details of the backend infrastructure.

Tasks

1. Propose a method to provide Access Transparency in the given architecture.
2. Implement your proposed solution.
3. Demonstrate that users can access the application without directly connecting to individual web servers.

Explain

- What component was used to hide backend complexity?
- How does the user interact with the system?
- Why is this considered Access Transparency?

Question 2 – Location Transparency

Users should not need to know where a service is physically located.

Tasks

1. Propose a method to hide the physical location of the web servers.
2. Test the application by sending multiple requests.
3. Observe the responses received from the backend servers.

Explain

- How is the server location hidden from users?
- Can the user determine which container handled a request?
- Why does your implementation demonstrate Location Transparency?

Question 3 – Replication Transparency

Multiple copies of the same service should exist while appearing as a single service to users.

Tasks

1. Propose a method to replicate the web service.

2. Deploy multiple identical web server containers.
3. Demonstrate that users interact with a single service despite multiple replicas existing.

Explain

- How many replicas were deployed?
- How are requests distributed among replicas?
- Why is replication hidden from the user?

Question 4 – Concurrency Transparency

Multiple users should be able to access the system simultaneously without affecting each other.

Tasks

1. Propose a method to support concurrent access.
2. Generate multiple simultaneous requests using multiple browser tabs or users.
3. Observe system behaviour.

Explain

- How does the system handle multiple requests?
- Were users able to access the service simultaneously?
- Why does this demonstrate Concurrency Transparency?